

differently: while Carolyn may think she is the only user on the computer, she really is not alone at all!

Attackers abuse exploits manually or automatically, to run malicious software or **malware**. Malware appears in different guises and there is a lot of confusion about terminology. We refer to malware that infects computers by injecting itself into other (often executable) files as a **virus**. In other words, a virus needs another program and typically some form of user interaction to propagate. For instance, the user should click on an attachment to get infected. In contrast, a **worm** is self-propelled. It will propagate regardless of what the user does. Well-known worms in the past would randomly scan IP addresses on the Internet to see if they found a machine with vulnerable software, and if so infect it, rinse, and repeat. A **Trojan**, or Trojan horse, is malware that is hidden in something that looks legitimate and/or useful. By repackaging popular but expensive software (like a game or a word processor) and offering it for free on the Internet, the attackers get users to install it themselves. For many users, “free” is completely irresistible. However, installing the free game automatically also installs additional functionality, the kind that hands over the PC and everything in it to a cybercriminal far away.

This chapter has two main parts. In the first part, we look at the topic of security in a principled manner. This includes the fundamentals of security (Sec. 9.1), different approaches to provide access control (Sec. 9.2), and formal models of secure systems (Sec. 9.3), which includes formal models for access control and cryptography. Authentication (Sec. 9.4) also belongs in this part.

So far, so good—in theory. Then reality kicks in and the second part introduces practical security problems that occur in daily life. We will talk about the tricks that attackers use to take control over a computer system using software vulnerabilities, as well as some common countermeasures to prevent this from happening (Sec. 9.5). Unfortunately, software bugs are no longer our only concern and we will briefly look at hardware vulnerabilities—for instance, cache side channels and speculative execution attacks (Sec. 9.6). However, even if the hardware and software are correct, there is still the human and we therefore briefly look at insider threats also (Sec. 9.7). Given the importance of security in operating systems, the security community has developed a variety of techniques to harden the operating system against attacks and we will review the most important ones (Sec. 9.8).

9.1 FUNDAMENTALS OF OPERATING SYSTEM SECURITY

Some people tend to use the terms “security” and “protection” interchangeably. Nevertheless, it is frequently useful to make a distinction between the general problems involved in making sure that files are not read or modified by unauthorized persons, which include technical, administrative, legal, and political issues, and the specific sets of rules maintained by the operating system to protect objects against unauthorized actions. To avoid confusion, we will use the term **security** to

refer to the overall problem, and the term **protection domain** to indicate the exact set of operations (such reading or writing a file or memory page) a user or process is permitted to perform on the objects in the system. Moreover, we will use **security mechanism** to refer to a specific technique used by the operating system to safeguard information in the computer. An example of a security mechanism is setting the supervisor bit in a pagetable entry of a page that should be inaccessible to user applications. Finally, we will use the term **security domain** in an informal way to refer to software that, on the one hand, needs to be able to perform its tasks securely, and on the other hand, needs to be prevented from jeopardizing the security of others. Examples of security domains include the components of the operating system kernel, processes and virtual machines. If we zoom in enough, we see that the notion of security domain is simply a convenient way to point at particular software. However, from a security point of view, all that is relevant about a security domain is defined by its protection domain.

9.1.1 The CIA Security Triad

Many security texts decompose the security of an information system in three components: confidentiality, integrity, and availability. Together, they are often referred to as “CIA.” They are shown in Fig. 9-1 and constitute the core security properties that we must protect against attackers and eavesdroppers—such as the (other) CIA.

The first, **confidentiality**, is concerned with having secret data remain secret. More specifically, if the owner of some data has decided that these data are to be made available only to certain people and no others, the system should guarantee that release of the data to unauthorized people never occurs. As an absolute minimum, the owner should be able to specify who can see what, and the system should enforce these specifications, which ideally should be per file.

Goal	Threat
Confidentiality	Exposure of data
Integrity	Tampering with data
Availability	Denial of service

Figure 9-1. Security goals and threats.

The second property, **integrity**, means that unauthorized users should not be able to modify any data without the owner’s permission. Data modification in this context includes not only changing the data, but also removing data and adding false data. If a system cannot guarantee that data deposited in it remain unchanged until the owner decides to change them, it is not worth much for data storage.

The third property, **availability**, means that nobody can disturb the system to make it unusable. Such **denial-of-service** attacks are increasingly common. For

example, if a computer is an Internet server, sending a flood of requests to it may cripple it by eating up all of its CPU time just examining and discarding incoming requests. If it takes, say, 100 μ sec to process an incoming request to read a Web page, then anyone who manages to send 10,000 requests/sec can wipe it out. Reasonable models and technology for dealing with attacks on confidentiality and integrity are available; foiling denial-of-service attacks is much harder.

Later on, people decided that three fundamental properties were not enough for all possible scenarios, and so they added additional ones, such as authenticity, accountability, nonrepudiability, and others. Clearly, these are all nice to have. Even so, the original three still have a special place in the hearts and minds of most (elderly) security experts.

9.1.2 Security Principles

While the challenges related to safeguarding these properties have also evolved over the past few decades, the principles have stayed mostly the same. For instance, when few people had their own computers and most computing was done on multiuser (often mainframe-based) computer systems with limited connectivity, security was mostly focused on isolating users or classes of users from each other. Isolation guarantees the separation of components (programs, computer systems, or even entire networks) that belong to different security domains or have different privileges. All interaction between the different components that takes place is mediated with proper privilege checks. Today, isolation is still a key ingredient of security. Even the entities to isolate have remained, by and large, the same. We will refer to them as security domains. Traditional security domains for operating systems are processes and kernels, and for hypervisors, virtual machines (VMs). Since then, some security domains (such as trusted execution environments) have been added to the mix, but these are still the main security domains today. There is no doubt, however, that the threats have evolved tremendously, and in response, so have the protection mechanisms.

While addressing security concerns in all layers of the network stack is certainly necessary, it is very difficult to determine when you have addressed them sufficiently and if you have addressed them all. In other words, *guaranteeing* security is hard. Instead, we try to improve security as much as we can by consistently applying a set of security principles. Classic security principles for operating systems were formulated as early as 1975 by Jerome Saltzer and Michael Schroeder:

1. **Principle of economy of mechanism.** This principle is sometimes paraphrased as the principle of simplicity. Complex systems always have more bugs than simple systems. Moreover, users may not understand them well and use them in a wrong or insecure way. Simple systems are good systems. This is also true for security solutions. One of the reasons Multics did not catch on as an operating system big time is that many users and developers found it cumbersome in practice.

Simplicity also helps to minimize the **attack surface** (all the points where an attacker may interact with the system to try to compromise it). A system that offers a large set of functions to untrusted users, each implemented by many lines of code, has a large attack surface. If a function is not really needed, leave it out. Back in the old days, when memory was expensive and scarce, programmers followed this principle out of necessity. The PDP-1 minicomputer, on which one of the authors cut his (programming) teeth, had 4K of 18-bit words or about 9 KB of total internal memory. It ran a timesharing system that could support three simultaneous users. Programs were necessarily very simple. Nowadays you cannot even boot the computer with less than a gigabyte of RAM and all this bloat makes programs buggy and unreliable.

2. **Principle of fail-safe defaults.** Say you need to organize the access to a resource. It is better to make explicit rules about how users can access the resource than trying to identify the condition under which access to the resource should be denied. Phrased differently: a default of lack of permission is safer. This is how locked doors work: If you don't have the key, you cannot get in.
3. **Principle of complete mediation.** Every access to every resource should be checked for authority. It implies that we must have a way to determine the source of a request (the requester).
4. **POLA (Principle of least authority).** This principle states that any (sub) system should have just enough authority (privilege) to perform its task and no more. Thus, if attackers compromise such a system, they elevate their privilege only by the bare minimum.
5. **Principle of privilege separation.** Closely related to the previous point: it is better to split up the system in multiple POLA-compliant components than a single component with all the privileges combined. Again, if one component is compromised, the attackers will be limited in what they can do.
6. **Principle of least common mechanism.** This principle is a little trickier and states that we should minimize the amount of mechanism common to more than one user and depended on by all users. Think of it this way: if we have a choice between implementing a file system routine in the operating system where its global variables are shared by all users, or in a user space library which, to all intents and purposes, is private to the user process, we should opt for the latter. The shared data in the operating system may otherwise serve as an information path between different users. We shall see examples of such sidechannels later.

7. **Principle of Open Design.** This states plain and simple that the design should not be secret and generalizes what is known as **Kerckhoffs' principle** in cryptography. In 1883, the Dutch-born Auguste Kerckhoffs published two journal articles on military cryptography that stated that a cryptosystem should be secure even if everything about the system, except the key, is public knowledge. In other words, do not rely on “security by obscurity,” but assume that the adversary immediately gains total familiarity with your system and knows the encryption and decryption algorithms. In modern terms, it means you should assume the enemy has the source code to the security system. Even better, you should publish it yourself to avoid fooling yourself into thinking it is secret. It is probably not.
8. **Principle of Psychological Acceptability.** The final principle is not a technical one at all. Security rules and mechanisms should be easy to use and understand. However, acceptability entails more. Besides the usability of the mechanism, it should also be clear why the rules and mechanisms are necessary in the first place.

9.1.3 Security of the Operating System Structure

The operating system is responsible for providing a foundation on which developers can build applications in their own security domains, and for protecting their confidentiality, integrity and availability. To do so, it uses the security principles of the previous paragraph—to isolate the security domains from each other and mediate all operations that may violate the isolation, and all the other nice things.

As we saw in Sec. 1.7, there are different ways to design an operating system. It turns out that the structure matters for security and some designs are inherently incompatible with some of the security principles. For instance, in early operating systems of the past, and many embedded systems today, there is no isolation whatsoever. All application and operating system functionality runs in a single security domain. In such a design, there is no notion of privilege separation or POLA.

Another important class of operating systems follows a monolithic design where most of the operating system resides in a single security domain, but isolated from the applications. The applications are also isolated from each other. Most general-purpose operating systems follow this design. Since the components of the operating system can interact using function calls and shared memory, it is very efficient. In addition, the design protects the more privileged parts of the system (the kernel of the operating system) from the less privileged parts (user processes). However, if attackers manage to compromise *any* component in the monolithic kernel, all security guarantees are toast.

Unfortunately, there is a lot of code there. Operating systems such as Linux and Windows consist of millions of lines of program code. A vulnerability in any of these lines could be fatal. This is bad enough for code that is part of the core operating system as that is generally carefully scrutinized for bugs, but it gets worse when we talk about device drivers and other extensions to the kernel. Often written by third parties, they tend to be more buggy than the core operating system code. If you buy a nifty new 3D printer, almost certainly you will have to download and install in your kernel a large piece of software about which you know nothing and which could contain many bugs and exploits. This should not be necessary.

An alternative design we discussed is to split up the operating system in many small components that each runs in a separate security domain. This is the approach taken by MINIX 3, shown in Fig. 1.26. Such a **multiserver operating system design** may have security domains for the process management code, the memory management functions, the network stack, the file system, and every driver in the system—with a very small microkernel running with higher privileges to implement the isolation at the lowest level. The model is less efficient than the monolithic one, because even the operating system components must now communicate with each other using IPC. On the other hand, adhering to the security principles is much easier. While a compromised printer driver may still embellish the pages coming out of the printer with hilarious messages and emojis, it is no longer able to pass the keys to the kingdom to the bad people.

Unikernels, finally, take yet another approach. Here, a minimal kernel is responsible only for partitioning the resources at the lowest level, but all operating system functionality required for the single application is implemented in the application's security domain in the form of a minimal “LibOS.” The design allows applications to customize the operating system functionality exactly according to their needs and leave out everything that they do not need anyway. Doing so reduces the attack surface. While you may object that running everything in the same security domain is bad for security, do not forget that there is only a single application in that domain—any compromise will effect only that application.

9.1.4 Trusted Computing Base

Let us dig into this a little deeper. In the security world, people often talk about **trusted systems** rather than secure systems. These are systems that have formally stated security requirements and meet these requirements. At the heart of every trusted system is a minimal **TCB (Trusted Computing Base)** consisting of the hardware and software necessary for enforcing all the security rules. If the trusted computing base is working to specification, the system security cannot be compromised, no matter what else is wrong.

The TCB typically consists of most of the hardware (except I/O devices that do not affect security), a portion of the operating system kernel, and most or all of the user programs that have superuser power (e.g., SETUID root programs in UNIX).

Operating system functions that must be part of the TCB include process creation, process switching, memory management, and part of file and I/O management. In a secure design, often the TCB will be quite separate from the rest of the operating system in order to minimize its size and verify its correctness.

An important part of the TCB is the reference monitor, as shown in Fig. 9-2. The reference monitor accepts all system calls involving security, such as opening files, and decides whether they should be processed or not. The reference monitor thus takes care of mediation, allowing all the security decisions to be put in one place, with no possibility of bypassing it. Most operating systems are not designed this way, which is part of the reason they are so insecure.

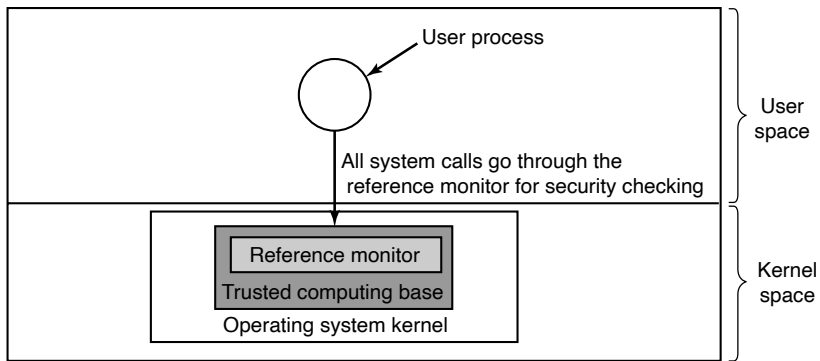


Figure 9-2. A reference monitor.

The TCB is closely related to the security principles we discussed earlier. For instance, a well-designed TCB is simple, separates privileges, applies the principle of least privilege, and so on. This brings us back to the design of the operating system. For some operating system designs, the TCB is enormous. In Windows or Linux, the TCB consists of all the code running in the kernel. That includes all the core functionality, but also all drivers. If you want to get picky, it also includes the compiler, since a rogue compiler could recognize when it is compiling the operating system and intentionally insert exploits in it that do not appear in the source code.

One of the goals of some current security research is to reduce the trusted computing base from millions of lines of code to merely tens of thousands of lines of code. Consider the MINIX 3 operating system: a POSIX-conformant system but with a radically different structure than Linux or FreeBSD. With MINIX 3, only about 15,000 lines of code run in the kernel. Everything else runs as a set of user processes. Some of these, like the file system and the process manager, are part of the TCB since they can easily compromise system security. But other parts, such as the printer driver and the audio driver, are not part of the trusted computing base and no matter what is wrong with them (even if they are taken over by a virus),

there is nothing they can do to compromise system security. By reducing the trusted computing base by two orders of magnitude, systems like MINIX 3 can potentially offer much higher security than conventional designs.

Unikernels may reduce the TCB in a different way, by aggressively removing everything that is not essential for the application in the Unikernel. While there may be little or no separation between the operating system and the kernel, the total amount of code in the TCB is much reduced compared to monolithic systems. The point is that the structure of the operating system has important consequences for the security guarantees.

9.1.5 Attackers

Systems are under constant threat from attackers who try to violate the security guarantees provided by the operating system or hypervisor, by stealing confidential data, modifying data to which they should not have access, or crashing the system.

There are many ways in which an outsider can attack a system; we will look at some of them later in this chapter. Many of the attacks nowadays are supported by highly advanced tools and services. Some of these tools are built by criminals, others by “ethical hackers,” who are trying to help companies find flaws in their software so they can be repaired before it is released.

Incidentally, the popular press tends to use the generic term “hacker” for the criminals. However, within the computer world, “hacker” is a term of honor reserved for all great programmers. While some of these are rogues, most are not. The press got this one wrong. In deference to true hackers, we will use the term in the original sense and will call people who try to break into computer systems where they do not belong either **crackers**.

Hackers and crackers have influenced operating system design in more ways than one. Not only have operating systems adopted a wide variety of protection mechanisms to prevent attackers from compromising the system, the crackers scene also served as a source of inspiration for the earlier computer pioneers. Steve Wozniak and Steve Jobs spent their time developing tools for phone phreaking (cracking the telephony system) before moving on to build a personal computer that they decided to call *Apple*. According to Wozniak, there would have been no Apple without John Draper, a controversial cracker popularly known as Captain Crunch. He acquired his nickname when he discovered that the toy whistle in a package of Cap’n Crunch cereals emitted a tone of 2600 Hz, which happened to be the exact frequency used by AT&T to authorize its (then expensive) long distance calls. Before founding one of the most successful computer companies in history, the two Steves similarly spent their time trying to get phone calls for free.

In the security literature, people who are nosing around places where they have no business being may also be called **attackers**, **intruders**, or sometimes **adversaries**. A few decades ago, cracking computer systems was all about showing your

friends how clever you were (and the authors will neither confirm nor deny rumors that they engaged in such activities in their younger days). Nowadays, however, this is no longer the only or even the most important reason to break into a system. There are many different types of attackers with different kinds of motivation. These include theft, hacktivism, vandalism, terrorism, cyberwarfare, espionage, spam, extortion, fraud—and occasionally the attacker still simply wants to show off, or expose the poor security of an organization.

Attackers similarly range from not very skilled beginners who want to be cybercriminals but have not learned the ropes yet, to extremely skillful crackers. They may be professionals working for criminals, governments (e.g., the police, the military, or the secret services), or security firms—or hobbyists that do all their hacking in their spare time. It should be clear that trying to keep a hostile foreign government from stealing military secrets is quite a different matter from trying to keep students from inserting a funny message-of-the-day into the system. The amount of effort needed for security and protection clearly depends on who the enemy is thought to be.

Going back to the attack tools, it may come as a surprise that many of them are developed by white hats. The explanation is that, while the baddies may (and do) use them also, these tools primarily serve as convenient means to test the security of a computer system or network. For instance, a **fuzzer** is a software testing tool that throws unexpected and/or invalid inputs at programs to see if the program hangs or crashes—evidence of bugs that should be fixed. Some fuzzers can only be used on simple user programs, but others explicitly target the operating system. A good example is Google's *syzkaller* which semi-randomly executes system calls with crazy combinations of arguments to trigger bugs in the kernel. Fuzzers may be used by developers to test their own code or corporate users to test the software they have purchased, but also by crackers who look for vulnerabilities that allow them to compromise the system. A tool that is useful for attackers as well as defenders is known as **dual use**. There are many examples of such tools.

However, cybercriminals also offer a wide range of (often online) services to wannabe cyber crooks who want to spread malware, launder money, redirect traffic, provide hosting with a no-questions-asked policy, and many other things that fit in the perceived business model. Most criminal activities on the Internet build on infrastructures known as **botnets** that consist of thousands (and sometimes millions) of compromised computers—often normal computers of innocent and ignorant users. There are all-too-many ways in which attackers can compromise a user's machine. While the hackers in movies typically “hack into” the system by exploiting some minuscule weakness in the victim's defenses with great genius (whatever that means), the reality may be more prosaic. For instance, they may guess the password, because “letmein” or “password” turn out to be less secure than many people think. The opposite also happens. Some users pick very complicated passwords, so that they cannot remember them, and have to write them down on a Post-it note which they attach to their screen or keyboard. This way, anyone

with physical access to the machine (including the cleaning staff, secretary, and all visitors) also has access to everything *on* the machine and also on all other machines to which that one has automatic access.

Another trick to infect somebody's computer is to offer free, but malicious versions of popular software that users install themselves: Trojans. Unfortunately, there are many other examples, and they include high-ranking officials losing USB sticks with sensitive information, old hard drives with trade secrets that are not properly wiped before being dropped in the recycling bin, and so on. None of these are terribly sophisticated.

Nevertheless, some of the most important security incidents *are* due to sophisticated cyber attacks. In this book, we are specifically interested in attacks that are related to the operating system. In other words, we will not look at Web attacks, or attacks on SQL databases. Instead, we focus on attacks where the operating system is either the target of the attack or plays an important role in enforcing (or more commonly, failing to enforce) the security policies. If you are interested in network security, there are many books on the topic, including Kaufman et al. (2022), Moseley (2021), Santos (2022), Schoenfield (2021), and Van Oorschot (2020).

In general, we distinguish between attacks that *passively* try to steal information and attacks that *actively* try to make a computer program misbehave. An example of a passive attack is an adversary that sniffs the network traffic and tries to break the encryption (if any) to get to the data. In an active attack, the intruder may take control of a user's Web browser to make it execute malicious code, for instance to steal credit card details. In the same vein, we distinguish between **cryptology**, which is all about shuffling a message or file in such a way that it becomes hard to recover the original data unless you have the key, and software **hardening**, which adds protection mechanisms to programs to make it hard for attackers to make them misbehave. The operating system uses cryptography in many places: to transmit data securely over the network, to store files securely on disk, to scramble the passwords in a password file, etc. Program hardening is also used all over the place: to prevent attackers from injecting new code into running software, to make sure that each process has exactly those privileges it needs to do what it is supposed to do and no more, etc.

When the computer is under control of the attacker, it is known as a **bot** or **zombie**. Typically, none of this is visible to the user. The attacker can use the bot to launch new attacks, steal passwords or credit card details, encrypt all the data on the disk for ransom, mine cryptocurrency, or any of the 1001 other things you can do with someone else's computer and electricity someone else is paying for.

Sometimes, the effects of the attack go well beyond the computer systems themselves and reach directly into the physical world. One example is the attack on the waste management system of Maroochy Shire, in Queensland, Australia—not too far from Brisbane. A disgruntled ex-employee of a sewage system installation company was not amused when the Maroochy Shire Council turned down his job application and he decided not to get mad, but to get even. He took control of the

sewage system and caused a million liters of raw sewage to spill into the parks, rivers and coastal waters (where fish promptly died)—as well as other places. A less smelly but perhaps dirtier and certainly scarier example of a cyberweapon interfering in physical affairs was *Stuxnet*—a highly sophisticated attack that damaged the centrifuges in a uranium enrichment facility in Natanz, Iran, and is said to have caused a significant slowdown in Iran’s nuclear program. While no one has come forward to claim credit for this attack, something that sophisticated probably originated with the secret services of one or more countries hostile to Iran.

9.1.6 Can We Build Secure Systems?

Nowadays, it is hard to open a newspaper without reading yet another story about attackers breaking into computer systems, stealing information, or controlling millions of computers. A naive person might logically ask two questions concerning this state of affairs:

1. Is it possible to build a secure computer system?
2. If so, why is it not done?

The answer to the first one is: “In theory, yes.” In principle, software and hardware can be free of bugs and we can even verify that it is secure—as long as that software or hardware is not too large or complicated. Unfortunately, computer systems today are horrendously complicated and this has a lot to do with the second question. The second question, why secure systems are not being built, comes down to two fundamental reasons. First, current systems are not secure but users are unwilling to throw them out. If Microsoft were to announce that in addition to Windows it had a new product, SecureOS, that was resistant to viruses but did not run Windows applications, it is far from certain that every person and company would drop Windows like a hot potato and buy the new system immediately.

In fact, Microsoft has had a highly secure operating system, Singularity, for years, but decided not to market it (Larus and Hunt, 2010). Since Windows 11 is based on a hypervisor, it would be easy to ship Windows 11 with two prebuilt virtual machines, Windows 11 and Singularity, and over a period of years gradually migrate security-sensitive applications to Singularity, but management decided not to do this for reasons best known to it.

The second issue is more subtle. The only known way to build a secure system is to keep it simple. Features are the sworn enemy of security. The good folks in the Marketing Dept. at most tech companies believe (rightly or wrongly, mostly wrongly) that what users desperately want is more features, bigger features, better features, sexier features, and ever more useless features. They make sure that the system architects designing their products get the word. However, all these mean more complexity, more code, more bugs, and more security errors.

One of the worst offenders is Apple, which is actually one of the tech companies that takes security extremely seriously. Apple devices have a feature called *handoff*, which allows you to start typing an email on a MacBook and then switch over to an iPhone to finish it. Did any user demand this before it was available? We doubt it. But Apple did it anyway, despite the large amount of potentially buggy code it required to implement this. So the downside of this more-or-less useless feature is thousands of lines of new code in the operating system, potentially with usable exploits, which also affect users who are not even aware of this feature, let alone who use it.

Here are two fairly simple examples. The first email systems sent messages as ASCII text. They were simple and could be made fairly secure. Unless there are really dumb bugs in the email program, there is little an incoming ASCII message can do to damage a computer system (we will actually see some attacks that may be possible later in this chapter). Then people got the idea to expand email to include other types of documents, for example, *Word* files, which can contain programs in macros. Reading such a document means running somebody else's program on your computer. No matter how much sandboxing is used, running a foreign program on your computer is inherently more dangerous than looking at ASCII text. Did users demand the ability to change email from passive documents to active programs? Probably not, but somebody thought it would be a nifty idea, without worrying too much about the security implications.

The second example is the same thing for Web pages. When the Web consisted of passive HTML pages, it did not pose a major security problem. Now that many Web pages contain programs (such as JavaScript) that the user has to run to view the content, one security leak after another pops up. As soon as one is fixed, another takes its place. When the Web was entirely static, were users up in arms demanding dynamic content? Not that the authors remember, but its introduction brought with it a raft of security problems. It looks like the Vice-President-In-Charge-of-Saying-No was asleep at the wheel.

Actually, there are some organizations that think good security is more important than nifty new features, the military being the prime example. In the following sections we will look some of the issues involved, but they can be summarized in one sentence. To build a secure system, have a security model at the core of the operating system that is simple enough that the designers can actually understand it, and resist all pressure to deviate from it in order to add new features.

9.2 CONTROLLING ACCESS TO RESOURCES

Security is easier to achieve if there is a clear model of what is to be protected and who is allowed to do what. Quite a bit of work has been done in this area, so we can only scratch the surface in this brief treatment. We will focus on a few general models and the mechanisms for enforcing them.